Quiz 1 - Bases de Datos

Universidad El Bosque

Programa: Ingeniería de Sistemas

Asignatura: Bases de Datos 2

Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

Quiz 2 — SQL Avanzado en Oracle: Joins, Subconsultas, CTE y Funciones Analíticas

Instrucciones. Duración: 40 minutos. No se permite consultar libros, apuntes ni dispositivos electrónicos. Responda con lápiz o esfero negro. Se evalúa la lógica de la consulta, no la digitación exacta; sin embargo, los errores de sintaxis que cambien el resultado o que impidan que la sentencia se ejecute en Oracle 19c se penalizan. Escriba de forma legible: lo que no se entiende no se califica.

Esquema de trabajo (común a las dos preguntas)

```
CLIENTE
(id_cliente PK, nombre, ciudad, segmento, fecha_registro)

SUCURSAL
(id_sucursal PK, nombre, ciudad, region)

CATEGORIA
(id_categoria PK, nombre,
 id_categoria_padre FK -> CATEGORIA,
 jerarquia de N niveles)

PRODUCTO
(id_producto PK, nombre, id_categoria FK, precio_lista)

PEDIDO
(id_pedido PK, id_cliente FK, id_sucursal FK,
 fecha_pedido, estado)

DETALLE_PEDIDO
(id_pedido PK, num_linea, id_producto FK,
 cantidad, precio_unitario, descuento)
PK (id_pedido, num_linea)

DEVOLUCION
(id_devolucion PK, id_pedido, num_linea,
```

```
  fecha_devolucion, cantidad_devuelta, motivo,
  FK (id_pedido, num_linea) -> DETALLE_PEDIDO)
```

Notas: un pedido puede tener varias devoluciones, incluso sobre la misma línea.

El descuento es un valor entre 0 y 1 (0.10 = 10%).

El valor de una línea es: cantidad * precio_unitario * (1 - descuento)

PREGUNTA 1 — Diagnóstico y reescritura de una consulta

Un desarrollador entrega la siguiente consulta. El requerimiento que debía cumplir es:

> «Para cada cliente del segmento PREMIUM, listar sus pedidos de 2025 con el valor total del pedido, el acumulado de compra del cliente hasta ese pedido inclusive, y el número de devoluciones asociadas al pedido —excluyendo las devoluciones por garantía—, mostrando únicamente los tres pedidos de mayor valor de cada cliente.»

```
SELECT
    c.id_cliente,
    c.nombre,
    p.id_pedido,
    p.fecha_pedido,
    SUM(d.cantidad* d.precio_unitario* (1- d.descuento))AS valor_pedido,
    SUM(SUM(d.cantidad* d.precio_unitario* (1- d.descuento)))
        OVER (PARTITION BY c.id_cliente
          ORDER BY p.fecha_pedido) AS acumulado,
    COUNT(dv.id_devolucion) AS num_devoluciones,
    RANK() OVER ( PARTITION BY c.id_cliente
                  ORDER BY SUM(d.cantidad* d.precio_unitario*
                  (1- d.descuento))DESC) AS puesto
    FROM cliente c
    JOIN pedido p ON p.id_cliente = c.id_cliente
    JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
    LEFTJOIN devolucion dv ON dv.id_pedido = p.id_pedido
    WHERE c.segmento='PREMIUM'
    AND dv.motivo<>'GARANTIA'
    AND EXTRACT(YEAR FROM p.fecha_pedido)=2025
    AND RANK() OVER ( PARTITION BY c.id_cliente
                      ORDER BY SUM(d.cantidad* d.precio_unitario*
                     (1- d.descuento))DESC)<=3
    GROUP BY
```

```
    c.id_cliente,
    c.nombre,
    p.id_pedido,
    p.fecha_pedido
    ORDER BY c.id_cliente, puesto;
```

a. (8 puntos)

Identifique **cuatro defectos** de la consulta. Para cada uno indique, en dos o tres líneas, qué está mal, por qué el resultado no cumple el requerimiento (o por qué ni siquiera se ejecuta) y cómo se corrige. Al menos uno de los defectos debe referirse al comportamiento de las funciones analíticas y al menos uno al comportamiento de los joins.

b. (4 puntos)

Reescriba la consulta completa y corregida usando **CTE (cláusula WITH)**. Debe cumplir exactamente el requerimiento enunciado, incluido el caso de los pedidos sin ninguna devolución, que deben aparecer con `num_devoluciones = 0`.

c. (3 puntos)

Considere estos pedidos del cliente 100 (segmento PREMIUM), con el valor del pedido ya calculado:

id_pedido	fecha_pedido	valor_pedido
9001	2025-03-04	500.000
9002	2025-03-04	1.200.000
9003	2025-03-11	300.000
9004	2025-04-02	800.000

Suponiendo que el resto de la consulta funcionara, indique qué valor devolvería la columna `acumulado` para cada uno de los cuatro pedidos, tal como está escrita. Explique en una línea a qué se debe la diferencia.

PREGUNTA 2 — Construcción de un informe gerencial

La gerencia comercial solicita un informe mensual de 2025 con las siguientes características:

- El informe se agrupa por **región de la sucursal**, por mes y por **categoría raíz del producto**. La categoría raíz es la categoría de primer nivel de la jerarquía (aquella cuyo `id_categoria_padre` es nulo) de la cual desciende la categoría del producto, que puede estar a cualquier profundidad.
- La venta neta es el valor vendido menos el valor de lo devuelto. El valor devuelto de una línea se calcula como `cantidad_devuelta * precio_unitario * (1 - descuento)`, tomando el precio y el descuento de la línea original, y se imputa al mes de la fecha de devolución, no al del pedido.
- Se debe mostrar la **variación porcentual de la venta neta** frente al mes inmediatamente anterior de la misma región y la misma categoría raíz. El primer mes de cada combinación debe quedar con datos de comparación.
- Se debe mostrar el **puesto de cada categoría raíz dentro de su región**, ordenado por la venta neta acumulada de todo 2025 de mayor a menor. Dos categorías con el mismo acumulado comparten puesto y no dejan huecos en la numeración.
- Solo se incluyen en el resultado las **tres categorías raíz de mayor venta neta anual de cada región**.
- Solo se consideran pedidos cuyo estado sea `'FACTURADO'`.

a.

Escriba una única sentencia `SELECT` que produzca el informe. Debe emplear obligatoriamente:

1. un **CTE recursivo o una consulta jerárquica** para resolver la raíz;
2. al menos dos funciones analíticas;
3. al menos un `OUTER JOIN` justificado.

Indique claramente el nombre y el propósito de cada CTE.

b.

Explique por qué la jerarquía de categorías no puede resolverse con un número fijo de self-joins y qué ocurre con su consulta si un día se agrega un nivel adicional a la jerarquía. Si usó CTE recursiva, escriba cuál es el miembro ancla y cuál el miembro recursivo; si usó `CONNECT BY`, indique qué hacen `START WITH` y `PRIOR` y cómo evitaría un ciclo infinito si por error se creara una categoría que es padre de sí misma.

c.

Un compañero propone reemplazar el filtro de las tres categorías por:

```
AND ROWNUM<=3
```

agregado al final del `WHERE`. Explique por qué esa solución es incorrecta en Oracle: qué devuelve realmente, en qué momento del procesamiento de la sentencia se asigna `ROWNUM` respecto al `ORDER BY`, y cuáles son las dos alternativas correctas para obtener un **top-N por grupo**.